

Ochorus

Web App Code Review

50 issues, bugs, and improvements — with a fix prompt for each

Django REST API • SvelteKit static SPA • multilingual reader
Reviewed 7 August 2026

How this review was made

Six independent review passes read the codebase in depth — the Django API, the content and translation pipeline, the frontend sync/offline libraries, the routes and components, a focused security audit across both stacks, and the CI/build/test infrastructure. Their ~98 raw findings were de-duplicated and curated down to the 50 below. Several were reproduced against a live test database (noted in place). Each finding states **what** it is, **why** it should be fixed, and a self-contained **fix prompt** you can paste into Claude Code.

Severity mix

CRITICAL **2** HIGH **13** MEDIUM **29** LOW **6**

Sections

A	Security & authentication	#1–7
B	Backend correctness & robustness	#8–14
C	Backend performance & API design	#15–21
D	Content & translation pipeline	#22–30
E	Frontend sync, offline & PWA	#31–38
F	Frontend UX, accessibility & SEO	#39–44
G	Engineering infrastructure & developer experience	#45–50

A. Security & authentication

The trust boundary between the Supabase JWT layer, the email-allowlist admin gate, and the public API.

01 Admin allowlist trusts the unverified email claim in the JWT

CRITICAL

backend/accounts/authentication.py:107; backend/accounts/permissions.py:44

WHAT

`_get_or_create_user` copies the token's email claim onto the Django user with no check of an `email_verified` claim, and `is_admin_user` grants admin purely on `user.email` in `ADMIN_EMAILS`. If the Supabase project has email confirmation disabled, anyone can register with the admin's email, receive a validly-signed token bearing it, and be treated as admin.

WHY IT MATTERS

Full takeover of every `/api/admin/*` endpoint (publish content, create authors/languages, trigger deploys, file GitHub jobs) by anyone who can obtain a signed token with the admin's address — no mailbox access required.

FIX PROMPT

In backend/accounts/authentication.py, after decoding the token reject or downgrade any JWT whose `email_verified` (or `user_metadata.email_verified`) claim is not truthy before trusting email for authorization. Ensure `permissions.is_admin_user` only considers a verified email, and add a test that an unverified-email token is never treated as admin even when the address is on `ADMIN_EMAILS`.

02 JWT decode does not verify the issuer and takes the algorithm from the token header

MEDIUM

backend/accounts/authentication.py:68

WHAT

`_decode` reads `alg` from the unverified token header and passes `algorithms=[alg]` with no issuer check. `exp` and `aud` are enforced and the HS secret is correctly sourced separately, but issuer is never validated and the audience check silently disables if `SUPABASE_JWT_AUDIENCE` is set empty.

WHY IT MATTERS

Defense-in-depth gap: a signature that validates against the configured keys but was minted for a different issuer/audience context is not rejected on those grounds.

FIX PROMPT

In backend/accounts/authentication.py, pass an explicit `issuer=f'{settings.SUPABASE_URL}/auth/v1'` to `jwt.decode` and require it, constrain `algorithms` to the specific expected set per key type (e.g. ES256/RS256 for JWKS, HS256 for the secret path) rather than echoing the token's `alg`, and fail closed when the audience setting is empty. Add tests for wrong-issuer and unexpected-alg tokens.

03 Open redirect via the login 'redirect' query parameter

MEDIUM

frontend/src/routes/login/+page.svelte:24

WHAT

`redirectTarget` is read straight from `$page.url.searchParams.get('redirect')` and passed to `goto()` after sign-in with no validation that it is a local same-origin path. A value like `//evil.com` or `https://evil.com` is honoured as a navigation target.

WHY IT MATTERS

A crafted `.../login?redirect=//evil.com` link sends a freshly-authenticated user to an attacker site, trading on the trust of the real login page (phishing / token relay).

FIX PROMPT

In frontend/src/routes/login/+page.svelte validate the `redirect` param before use: accept it only if it begins with a single `/` and not `//`, otherwise fall back to `localizeHref('/')`. Add a unit test covering `//evil.com`, `https://evil.com`, and a valid `/books/x` path.

04 DRFBrowsableAPI renderer is enabled in production

LOW

backend/config/settings.py:140

WHAT

DEFAULT_RENDERER_CLASSES includes BrowsableAPIRenderer unconditionally, so every endpoint — including /api/admin/* — serves an interactive HTML explorer with forms and serializer metadata to any browser sending Accept: text/html, in production.

WHY IT MATTERS

Unnecessary attack surface and information disclosure: it advertises endpoint shapes and forms to probers and serves no purpose to a JSON-only SPA.

FIX PROMPT

In backend/config/settings.py set DEFAULT_RENDERER_CLASSES to JSONRenderer only, then append BrowsableAPIRenderer after the DEBUG flag is computed when DEBUG is true. Verify the admin dashboard and frontend still work against the JSON-only renderer.

05 Django admin mounted at the predictable /admin/ with no rate limiting

LOW

backend/config/urls.py:35

WHAT

The full Django admin (admin.site.urls) is mounted at /admin/ with session auth separate from the Supabase JWT layer and no brute-force protection. The project otherwise has no is_staff concept, so this is a distinct, easily-found credential-login surface.

WHY IT MATTERS

A discoverable password-login endpoint invites credential-stuffing against any superuser account; a compromise there is unrestricted DB access.

FIX PROMPT

In backend/config/urls.py move the Django admin to an env-configurable non-obvious path and add login rate limiting (e.g. django-axes) or a network restriction. Confirm no automation depends on the /admin/ path before moving it.

06 Publish endpoint stores external URLs with only a scheme-prefix check

LOW

backend/library/upload_import.py:218

WHAT

_http_url accepts any string starting with http:// or https:// and stores it as source_url/cover_url; these are later emitted to readers as . There is no host allowlist and non-TLS http:// is permitted. The values are not fetched server-side (so not SSRF), but they are unvalidated stored URLs on an admin write.

WHY IT MATTERS

A stored http:// cover causes mixed-content on the HTTPS reader, and an unconstrained external cover host can track readers via image requests.

FIX PROMPT

In backend/library/upload_import.py tighten _http_url to require https:// and optionally validate the host against an allowlist (or store covers locally). Ensure cover_url/source_url render safely on the frontend. Add tests for rejected schemes/hosts.

07 Popular Searches echoes raw reader query text publicly

LOW`backend/library/views.py:512`

WHAT

PopularSearchesView is AllowAny and returns the raw lowercased text of reader queries above frequency thresholds (MIN_COUNT>=5, MIN_DISTINCT>=4). The privacy protection is purely structural; the endpoint reflects reader-entered strings verbatim to anyone and, combined with the unthrottled logging below, is seedable.

WHY IT MATTERS

On low-traffic locales a handful of repeated queries can surface partial reader input publicly.

FIX PROMPT

Review PopularSearchesView in backend/library/views.py: consider an admin-only or curated-allowlist mode and ensure the frequency thresholds are high enough on production traffic that no low-volume reader text is echoed. Add a test asserting sub-threshold queries never appear.

B. Backend correctness & robustness

The reading-sync merge path — the reconciliation that must never lose or corrupt a reader's offline work.

08 Merge and PUT endpoints 500 with AttributeError on malformed JSON

HIGH

backend/reading/views.py:319

WHAT

`_merge_progress`, `_merge_favorites`, `_merge_marks` and `_merge_sermon_marks` iterate `request.data.get(...)` or `[]` and immediately call `row.get(...)` without checking the container is a list or the row is a dict — only `_merge_plan_progress` and `_merge_activity` have those guards. Verified: `{'progress':'oops'}`, `{'favorites':[42]}`, `{'marks':[42]}` and a bare JSON array body all raise `AttributeError` -> 500.

WHY IT MATTERS

The first-sign-in merge is the critical reconciliation path; one malformed field in a client bundle (a stale `localStorage` shape, a buggy PWA build) aborts the whole merge with a 500 instead of skipping the bad rows.

FIX PROMPT

In backend/reading/views.py harden every `_merge_*` helper: return early when incoming is not a list and continue on non-dict rows (mirroring the existing guards in `_merge_plan_progress`); in `MarksView.put`, `ProgressView.put` and `SermonMarksView.put` treat a non-dict `request.data` as `{}` like `SearchClickView` does. Add tests posting a string, an int-list and an array body to each endpoint asserting no 500.

09 Unbounded merge fan-out — authenticated write-amplification with no throttle

HIGH

backend/reading/views.py:357

WHAT

`MAX_ACTIVITY_MERGE` caps only `_merge_activity`. Favorites, marks, progress, sermon-marks and plan-progress iterate the full incoming list with no cap, issuing 1-3 queries per row, and slugs are arbitrary strings so every row creates a new DB row. No `/api/reading/` endpoint carries a throttle.

WHY IT MATTERS

Any signed-up user can send one POST with a million favorite rows and drive millions of sequential queries on a Render worker, or grow the tables without limit.

FIX PROMPT

In backend/reading/views.py cap every merge list to a per-type maximum (reuse the `MAX_ACTIVITY_MERGE` pattern), convert `_merge_activity/_merge_favorites` to `bulk_create(ignore_conflicts=True)`, and add a per-user throttle scope for the reading endpoints in `settings.REST_FRAMEWORK.DEFAULT_THROTTLE_RATES` applied to `MergeView` at minimum. Add a test asserting an oversized bundle is truncated, not fully written.

10 MergeView is not atomic — a mid-merge failure leaves half-applied state

MEDIUM

`backend/reading/views.py:319`

WHAT

`MergeView.post` runs six sequential merge phases, each many writes, with `no transaction.atomic`. If any phase raises, earlier phases are already committed while later ones never ran, and the client receives a 500 instead of the merged state it was about to write back over `localStorage`.

WHY IT MATTERS

A partially-merged account is the worst outcome for the 'nothing a reader did offline is lost' contract: the client may treat the 500 as sync failure while the server already holds a different half of the truth.

FIX PROMPT

In `backend/reading/views.py` wrap the body of `MergeView.post` (all six `_merge_*` calls) in `django.db.transaction.atomic()` so reconciliation is all-or-nothing, and add a test that a payload failing in a later phase leaves no partial favorites written.

11 Read-modify-write race in `_upsert_plan_progress` can drop completed plan days

MEDIUM

`backend/reading/views.py:91`

WHAT

`_upsert_plan_progress` reads the existing row with `.first()`, unions done in Python, then writes via `update_or_create`. Two devices PUTting concurrently — the exact scenario the docstring claims to handle — can both read the same old done and the second write clobbers the first's union, losing a completed day.

WHY IT MATTERS

The module's whole merge contract ('a completion earned on any device is never lost') is violated under concurrency.

FIX PROMPT

In `backend/reading/views.py` make `_upsert_plan_progress` atomic: wrap in `transaction.atomic()` and fetch the existing row with `select_for_update()` before unioning. Keep the union-of-done / earliest-start semantics and add a comment noting the locking backs the 'never lost' guarantee.

12 Negative `chapter_order` clamps to 0 and is stored instead of rejected

MEDIUM

`backend/reading/views.py:397`

WHAT

`_merge_marks` uses `order = _clamp_int(row.get('chapter_order'), default=-1, low=0)` then skips rows with `order < 0`. But `_clamp_int` returns `max(low, int(value))`, so a parseable -5 becomes 0 — the sentinel only fires for unparseable values. Verified: a `chapter_order=-5` payload stores a `ChapterMarks` row with `chapter_order=0`, but chapters are 1-based so chapter 0 is a junk row.

WHY IT MATTERS

The intended validation is dead code for negative numbers; corrupt client data lands as orphan rows that surface in `_serialize_state` and confuse the client's rehydration.

FIX PROMPT

In `backend/reading/views.py` `_merge_marks` replace the clamp with explicit parsing: `try order = int(row.get('chapter_order'))` and continue unless `order >= 1`. Add a test asserting a negative `chapter_order` row is skipped, not stored as order 0.

13 Over-long slugs and huge chapter orders cause Postgres DataError 500s

MEDIUM

backend/reading/urls.py:19; backend/reading/views.py:255

WHAT

Favorite.slug and the two book_slug fields are SlugField(max_length=160), but Django's <slug:> converter accepts unbounded length and MergeView takes slugs straight from JSON with no length check; <int:order> accepts values above PositiveIntegerField's limit. On Postgres a 300-char slug PUT raises DataError -> 500 (SQLite dev masks this).

WHY IT MATTERS

Trivially crafted requests turn into 500s (Sentry noise) instead of clean 400s, and mid-merge they trigger the partial-write problem above.

FIX PROMPT

In backend/reading/views.py truncate or reject over-long identifiers before writing (slice slugs to 160 chars or return 400 when longer) across FavoriteView, ProgressView, MarksView, SermonMarksView and every MergeView _merge_ helper; bound order/chapter_order to a sane maximum. Add a test PUTting a 300-char slug asserting a non-500 response.*

14 Highlight note length and mark count per chapter are unbounded

MEDIUM

backend/reading/marks.py:35

WHAT

clean_mark_list validates each mark's shape but bounds neither the number of marks per chapter nor the length of note or id. One authenticated PUT can store a multi-megabyte JSON blob in a single ChapterMarks.marks field, and because book_slug is any string, unlimited junk rows can be minted. merge_mark_lists then unions these on every merge and _serialize_state returns them all on every sign-in.

WHY IT MATTERS

Storage abuse plus a self-inflicted payload bomb: StateView/MergeView responses balloon with the stored data, slowing every sync for that account.

FIX PROMPT

In backend/reading/marks.py clean_mark_list cap the accepted list (e.g. first 500 valid marks), truncate note to ~5000 chars and id to ~64 chars, and document the bounds; apply the same cap after merge_mark_lists in views.py. Add tests for an oversized mark list and an oversized note.

C. Backend performance & API design

Query efficiency and payload consistency on the prerender-time hot paths (author, book, plan, search).

15 word_count silently missing from book cards everywhere except the /books shelf MEDIUM

backend/library/serializers.py:123

WHAT

BookListSerializer.word_count = IntegerField(source='total_words') depends on a total_words annotation. AuthorDetailSerializer._books annotates only num_chapters, so DRF drops the field via SkipField. Verified: author-detail book cards contain chapter_count but no word_count key, while other paths include it — the same serializer emits two different shapes.

WHY IT MATTERS

Reading-time estimates silently disappear on author pages, and the inconsistent payload is a trap for every future consumer.

FIX PROMPT

In backend/library/serializers.py add total_words=Sum('chapters__word_count') to the annotation in AuthorDetailSerializer._books, and audit every queryset serialized with BookListSerializer for both annotations. Add a test asserting author-detail book cards each contain a word_count key.

16 Book topic chips are always empty outside BookListView MEDIUM

backend/library/serializers.py:143

WHAT

BookListSerializer.get_topics returns self.context.get('book_topics', {}).get(obj.slug, []). That map is built only in BookListView.get_serializer_context, so when the same serializer renders books inside AuthorDetailSerializer, TopicDetailSerializer and BookDetailSerializer.get_related, topics is silently [] for every book.

WHY IT MATTERS

Author pages, topic pages and 'more like this' cards can never show topic chips — a quiet feature loss the API can't signal (empty list is indistinguishable from 'no topics').

FIX PROMPT

In backend/library/serializers.py make get_topics fall back to a real lookup when book_topics is absent (call the existing _topic_chips helper with the resolved language), or build and thread the book_topics map in AuthorDetailView and TopicDetailView contexts as BookListView does. Add a test that an author-detail response includes topic chips for a book in a published topic.

17 AuthorDetailSerializer re-executes the same book/sermon queries up to five times MEDIUM

backend/library/serializers.py:334

WHAT

_books(obj) returns a fresh queryset each call and is evaluated by get_books (full fetch), get_book_count (.count()) and get_topics (.values_list) — three round-trips over the same filter; _sermons(obj) is called by both get_sermon_count and get_sermons, and get_topics re-queries the author's sermons a third way.

WHY IT MATTERS

The author page — a prerender-time hot path across every locale — pays ~6 queries where 2 would do, multiplied by authors x languages on each build.

FIX PROMPT

In backend/library/serializers.py AuthorDetailSerializer memoize per instance: cache list(self._books(obj)) and self._sermons(obj), derive the counts via len(), and derive the slug sets in get_topics from the cached lists. Add an assertNumQueries test for the author-detail endpoint.

18 BookDetailView loads every chapter's full body just to render a TOC

MEDIUM

`backend/library/views.py:181`

WHAT

`BookDetailView.get_object` does `.prefetch_related('chapters', ...)`, loading complete Chapter rows — `body_html`, `body_text` and the `search_vector` `tsvector` — for every chapter, when the serializer needs only `order/title/word_count` for the TOC and the first five chapters' `body_text` for `get_difficulty`.

WHY IT MATTERS

For a long book (Confessions, Pilgrim's Progress) this drags megabytes from Postgres into memory per book-page request and per prerender, for data that is thrown away.

FIX PROMPT

In backend/library/views.py replace the bare chapters prefetch with `Prefetch('chapters', queryset=Chapter.objects.defer('body_html','search_vector'))`, keeping `body_text` for `get_difficulty` (or defer it too and have `get_difficulty` run its own `.only('body_text')` query over the first five chapters). Verify the book-detail test still passes.

19 SearchView performs an unthrottled anonymous DB write on every request

MEDIUM

`backend/library/views.py:476`

WHAT

Every unscoped search creates a `SearchQueryLog` row with no throttle on `SearchView` (only `SearchClickView` is throttled). It is one row per request, per keystroke, capped only by a 180-day trim, and every zero-result query additionally runs the full-vocabulary `difflib` suggest scan.

WHY IT MATTERS

Anyone can script millions of `?q=` requests to grow the table and skew the popular-searches analytics that drive content decisions, at zero cost.

FIX PROMPT

In backend/library/views.py add a throttle to `SearchView` analogous to `_SearchClickThrottle` (a `UserRateThrottle` subclass with its own generous scope, e.g. 120/min for search-as-you-type, registered in settings), or at minimum bound the logging behind a per-IP token bucket. Preserve fail-open behaviour and add a test that the limit is enforced for anonymous callers.

20 Scripture-reference searches fetch every sermon's full body into Python

LOW

`backend/library/search.py:719`

WHAT

`_scripture_sermon_hits` iterates full sermon instances (including `body_html` and `body_text`) for every published sermon in the language and calls `reference_verse_ids` per row, even though only the winners' `body_text` is used — while the neighbouring `_scripture_chapter_hits` explicitly avoids exactly this ('measured 40 MB').

WHY IT MATTERS

Every query that parses as a scripture reference ('John 3:16') loads the whole sermon corpus' bodies, per keystroke once digits appear.

FIX PROMPT

In backend/library/search.py `_scripture_sermon_hits` first iterate `.values('slug','scripture_ref')` to pick winning slugs by verse-id overlap, then fetch only the winners with `.defer('body_html','search_vector')`, mirroring `_scripture_chapter_hits` two-phase pattern. Keep dedupe and the sermon cap identical.

21 No DRF pagination configured; all list endpoints return unbounded result sets

LOW

backend/config/settings.py:133; backend/library/views.py:343

WHAT

REST_FRAMEWORK sets no DEFAULT_PAGINATION_CLASS, so author/book/sermon/plan list views return every row in one response. TopicListView even sets pagination_class = None explicitly, a no-op given the default is already None — evidence the codebase believes pagination exists somewhere.

WHY IT MATTERS

As the sermon/book count grows into the hundreds per language, shelf payloads and query time grow linearly with no pressure valve, and the meaningless pagination_class = None misleads future readers.

FIX PROMPT

In backend/config/settings.py either add a generous DEFAULT_PAGINATION_CLASS/PAGE_SIZE (LimitOffsetPagination with a high default limit so static-build consumers are unaffected) or, if unpaginated lists are a deliberate prerender contract, document that in a comment on REST_FRAMEWORK and delete the dead pagination_class = None from TopicListView.

D. Content & translation pipeline

The seed/upsert and AI-translation machinery, and the invariant that unreviewed text is never presented as approved.

22 Translating new chapters into an approved book silently keeps it 'reviewed' CRITICAL

backend/library/management/commands/translate_book.py:86

WHAT

The book row's `source_type` is only written inside if target is None or force. If a translation was approved (`ai_reviewed`) and the English source later gains chapters (or `--chapters` targets a missing order), the chapter loop inserts brand-new machine-translated chapters into the approved book without touching `source_type`. The same hole exists in `contemporize_book`.

WHY IT MATTERS

Unreviewed AI text is presented to readers without the 'awaiting native review' badge — a direct violation of the never-present-unreviewed-as-approved invariant.

FIX PROMPT

In `translate_book.py` and `contemporize_book.py`, after the chapter loop, if any chapter was created or updated and the target's `source_type` is `AI_REVIEWED`, flip it back to `AI_UNREVIEWED` and save (printing that re-review is required). Add a regression test: an approved target, one new chapter via a mocked client, assert the book is `ai_unreviewed` afterwards.

23 `seed_plans` is not atomic and never repairs a plan created without its days HIGH

backend/library/management/commands/seed_plans.py:277

WHAT

`handle()` has no `transaction.atomic`, and `Plan.objects.create(...)` is followed by a separate `PlanDay.objects.bulk_create(...)`. A crash between the two leaves a day-less plan; on every later deploy `_reconcile_existing` returns True for any existing (slug, language) row without checking it has days, so creation is skipped forever.

WHY IT MATTERS

A single mid-seed failure produces a permanently empty reading plan visible to readers, and re-running the 'idempotent' command can never fix it.

FIX PROMPT

In `backend/library/management/commands/seed_plans.py` wrap each plan's `create+bulk_create` in one atomic block, and make `_reconcile_existing` treat a plan with zero `PlanDays` as 'not existing' (backfill or delete-and-recreate its days). Add a test that a Plan with no days is repaired by `seed_plans`.

24 Re-importing a book re-asserts is_published / source_type, republishing pulled works

HIGH

backend/library/management/commands/import_ochorus.py:535

WHAT

upsert() puts is_published: bool(chapters) and source_type: PUBLIC_DOMAIN in the defaults of update_or_create, so any re-import overwrites them. EXCLUDED_SLUGS only guards catalogue-wide imports; an explicit import_ochorus <slug> (allowed per the docstring) recreates and republishes a book that the copyright audit had unpublished.

WHY IT MATTERS

One command invocation re-publishes non-public-domain content, and a later fixture serialization would ship is_published: true to every fresh install.

FIX PROMPT

In import_ochorus.py upsert() (and ingest.upsert_book) move is_published and source_type out of the update defaults into create_defaults so re-imports never republish or re-type an existing row; make import_ochorus refuse slugs in corrections.EXCLUDED_SLUGS unless an explicit --include-excluded flag is passed. Add a test that re-importing an unpublished book leaves is_published False.

25 Translation approvals live only in the prod DB and are lost on a fresh-DB rebuild

HIGH

backend/library/management/commands/approve_translation.py:29

WHAT

approve_translation / approve_sermon_translation / the admin approve endpoint flip source_type on the live row only; the fixture still says ai_unreviewed (seed_books' own comment notes '37 fixture books still say ai_unreviewed'), and regen_fixture.py regenerates the fixture from the fixture, never from a DB, so the flip can never round-trip into the committed files.

WHY IT MATTERS

Any fresh install or disaster-recovery rebuild silently re-gates every approved translation back to 'awaiting native review', losing months of review decisions.

FIX PROMPT

Make approvals durable in the repo: have approve_translation (and the sermon/admin variants) also apply the source_type edit to the book's fixture file, or add a small committed approvals list that a release step re-applies after seed_if_empty. Add a test that a fresh loaddata + release preserves a recorded approval.

26 fetch_chapter permanently negative-caches transient failures, dropping scripture

HIGH

backend/library/translation.py:107

WHAT

On any RequestException or non-OK response, _verse_cache[key] = None is stored and never retried for the life of the process. scripture_context fails open by skipping None passages, so after one network blip mid-translate_book every later chapter citing that passage is translated with no authoritative verse text — despite verify_bible_code passing at job start.

WHY IT MATTERS

The exact failure the module's docstring calls the worst case (model-invented scripture wording, at full cost, looking entirely normal) can happen from a single transient timeout in a long paid job.

FIX PROMPT

In backend/library/translation.py fetch_chapter distinguish a definitive miss (HTTP 404) from a transient failure: cache only 404s, and retry transient failures with small backoff, raising after repeated failure so the job stops rather than shipping scripture-less chapters. Add a test with a mocked requests.get that fails once then succeeds, asserting the retry.

27 Integrity audit keys chapters by slug only, merging all language editions

MEDIUM

backend/library/admin_views/quality.py:177

WHAT

`_scan_chapters` builds titles and orders dicts keyed by book `__slug`, but a slug has one row per language. `_order_gaps` takes `set(orders)` across all editions, so a Spanish edition missing chapter 3 is invisible whenever the English edition has one; `_duplicate_titles` counts a title repeated across two language editions as a duplicate within one book.

WHY IT MATTERS

The audit can never detect a chapter gap in a partially-translated book (the most likely place for one) and reports false duplicate-title findings that erode trust in the report.

FIX PROMPT

In backend/library/admin_views/quality.py `_scan_chapters` key the accumulators by (slug, language) and carry language through the `_duplicate_titles` / `_order_gaps` output rows. Add tests: an es edition missing an order the en edition has is reported; en and es sharing a title string is not a duplicate.

28 Admin publish accepts any language string and stamps non-English uploads public_domain

MEDIUM

backend/library/admin_import_views.py:107

WHAT

`AdminImportPublishView` takes language from the payload with no validation against `languages.known_codes()` (the jobs endpoint validates) and no normalization, and `create_book/create_sermon` hard-code `source_type=PUBLIC_DOMAIN` regardless of language.

WHY IT MATTERS

A typo ('Sw', 'es ') strands content in a language no reader can reach, and a human- or third-party-translated upload is labelled a public-domain original with no review pathway.

FIX PROMPT

In backend/library/admin_import_views.py `AdminImportPublishView.post` lowercase/strip language and reject codes not in `library.languages.known_codes()` with a 400; consider accepting an optional `source_type` restricted to `Book.SourceType`. Add tests for an unknown code and a mixed-case code.

29 TopicTranslation has no review state despite the pipeline's review-flow promise

MEDIUM

backend/library/models.py:587

WHAT

`seed_topics` comments say topic prose is 'AI-drafted, pending native review (the same review flow as book translations)', but `TopicTranslation` has no `reviewed/source_type` field, no badge, and no approve command; the `seed`'s `update_or_create` re-asserts the code dict over any DB edit every deploy.

WHY IT MATTERS

AI-drafted shelf prose is presented indistinguishably from reviewed text — the one content type that escapes the `ai_unreviewed` -> approval invariant — and a native reviewer has no way to record or protect an approval.

FIX PROMPT

Add a reviewed BooleanField (default False) to `TopicTranslation` with a migration; make `seed_topics` skip overwriting rows with `reviewed=True` (mirroring `seed_author_translations`' ownership rule), add an approve path (extend the review queue and/or an `approve_topic_translation` command), and surface unreviewed topics in the queue. Pin with tests.

30 **apply_body_corrections runs before the content seeds, so new rows ship uncorrected**

MEDIUM

backend/library/management/commands/release.py:32

WHAT

The release chain runs `apply_body_corrections` before `seed_books/seed_sermons`. A work first created by this deploy's seed (a new fixture whose text still carries a defect in `BODY_CORRECTIONS`) is created after the correction pass and ships uncorrected until the next deploy.

WHY IT MATTERS

Prod serves known-defective text — the exact strings the corrections table exists to fix — for one deploy window on every new-work release, with nothing reporting it.

FIX PROMPT

In backend/library/management/commands/release.py move `apply_body_corrections` to run after `seed_books` and `seed_sermons` (it is idempotent), or run it a second time after the seeds. Add a release-ordering test that seeds a new fixture book containing a correctable string and asserts the stored chapter is corrected within the same run.

E. Frontend sync, offline & PWA

The localStorage-first stores, the sign-in merge, the service worker, and the shared-device data boundary.

31 Service worker can permanently cache authenticated /api/ responses (cross-user leak)

HIGH

frontend/src/service-worker.ts:100

WHAT

The fetch handler routes every same-origin GET that isn't a navigation, library API, or image to cacheFirst, which caches any ok response forever within a deploy and never revalidates. config.ts documents API_BASE_URL="" (same origin) as the production mode; under that config /api/auth/me/ is cached on first fetch, and the cache key ignores the Authorization header.

WHY IT MATTERS

Profile pulls go permanently stale within a deploy, and on a shared browser a second user is served the previous user's cached /api/auth/me/ response. It only doesn't bite today because render.yaml happens to set a cross-origin API URL, contradicting config.ts's own comment.

FIX PROMPT

In frontend/src/service-worker.ts add an early bail for any pathname starting with /api/ that isn't matched by isLibraryApi, so authenticated endpoints go straight to the network and are never cached. Reconcile the config.ts comment with render.yaml's cross-origin PUBLIC_API_BASE_URL.

32 Session-expiry sign-out skips the shared-device data wipe

HIGH

frontend/src/lib/auth.svelte.ts:66

WHAT

clearOnSignOut() runs only inside the explicit signOut() method. When Supabase fires SIGNED_OUT through onAuthStateChange for any other reason (refresh token expired/revoked, signed out in another tab, password changed elsewhere), #applySession(null) clears the token and user but leaves all reading data in localStorage.

WHY IT MATTERS

This defeats the exact scenario the wipe exists for: on a shared device, user A's expired session leaves their highlights/notes/progress behind, and user B's next sign-in merge uploads A's private reading data into B's account permanently.

FIX PROMPT

In frontend/src/lib/auth.svelte.ts, in the onAuthStateChange callback detect the signed-in -> signed-out transition (wasSignedIn && !session) and call readingSync.clearOnSignOut() plus reset displayName/isAdmin, mirroring signOut(). Keep the removal idempotent so the explicit signOut() path doesn't double-wipe.

33 Bookmarks are wiped on sign-out but never synced — guaranteed data loss

HIGH

`frontend/src/lib/bookmarks.svelte.ts:4; frontend/src/lib/reading-schema.ts:39`

WHAT

BOOKMARKS_KEY is in READING_DATA_KEYS, so `clearDeviceData()` deletes it on sign-out, but `readingSync` has no `pushBookmarks`, the merge payload omits bookmarks, and the server state never restores them. Marks/progress/favorites all round-trip through the account; bookmarks are the only store wiped without a server copy.

WHY IT MATTERS

A signed-in reader who signs out permanently loses every explicit bookmark with no warning; the settings copy ('the next sync restores from the account') is false for this data class.

FIX PROMPT

Either add bookmarks to the sync layer (a debounced `pushBookmarks` PUT, bookmarks in the merge payload and response, plus the matching Django endpoint/model), or as a stopgap remove BOOKMARKS_KEY from READING_DATA_KEYS so sign-out stops destroying the only copy, noting the shared-device tradeoff in the comment.

34 Merge response clobbers local mutations made while the request is in flight

MEDIUM

`frontend/src/lib/readingSync.ts:267`

WHAT

`mergeOnSignIn()` snapshots `localStorage`, POSTs it, and on response `#writeState` unconditionally overwrites `progress/marks/favorites/activity/plans` with the server's answer. It runs on every full page load for a signed-in user, so a highlight, favorite, or chapter-open recorded between snapshot and response is erased locally when the response lands. There is also no guard against two concurrent merges interleaving.

WHY IT MATTERS

The reader sees their just-made highlight or 'continue reading' position visibly revert; the debounced push may still deliver it, leaving local and server silently divergent.

FIX PROMPT

In `frontend/src/lib/readingSync.ts` add an in-flight guard (a shared promise) and, instead of blind overwrite, re-merge the response with current `localStorage` (newest-at wins for progress; union marks/favorites/activity/plan days), or detect that `localStorage` changed since the snapshot and re-run the merge.

35 Highlights are keyed without language, corrupting every translation of a work

MEDIUM

`frontend/src/lib/reading-schema.ts:82`

WHAT

`workKey(kind, slug, order)` omits language, so marks made in the English edition are loaded verbatim when the reader opens the Spanish or en-modern edition of the same chapter — but marks are character offsets into paragraph text that differ per translation. The merge payload rows also omit language even though `pushMarks` sends it.

WHY IT MATTERS

Readers of any translated or modernized edition see highlights attached to the wrong spans (or mid-word), and the server's language column is unreliable after a merge.

FIX PROMPT

In `frontend/src/lib/reading-schema.ts` extend `workKey/parseWorkKey` with a language component for non-'en' entries (keeping bare keys for existing 'en' data), thread language through `marks.svelte.ts` `load/persist`, and include language in the `mergeOnSignIn` marks rows to match `pushMarks`. Coordinate with the Django merge endpoint's identity columns.

36 apiFetch has no timeout or retry; every sync push is fire-and-forget with silent drop

MEDIUM

`frontend/src/lib/api.ts:28; frontend/src/lib/readingSync.ts:136`

WHAT

apiFetch calls bare fetch with no AbortController/timeout, so a hung connection leaves promises pending indefinitely — including auth.init's awaited profile/merge chain. All readingSync push* calls end in `.catch(() => {})`: a failed push is dropped with no retry and no dirty flag.

WHY IT MATTERS

In a long-lived PWA session (the app's core offline audience), hours of highlights and progress can silently fail to mirror, and users on flaky networks get UI that waits forever.

FIX PROMPT

In frontend/src/lib/api.ts wrap apiFetch/apiFetchRaw with an AbortController timeout (~15s, overridable). In readingSync.ts, on push failure set a persisted dirty flag and re-run the push (or a full merge) when the 'online' event fires (pwa.svelte.ts already tracks connectivity) instead of swallowing the error.

37 A book with failed chapter downloads is still recorded as fully available offline

MEDIUM

`frontend/src/lib/offlineBooks.svelte.ts:82`

WHAT

In `download()`, each chapter fetch failure is swallowed by the inner try/catch or the `res.ok` check, the progress counter still advances, and after the loop the book is unconditionally appended to the downloaded list with `chapterCount: orders.length`. Nothing tracks how many URLs actually made it into the cache.

WHY IT MATTERS

The UI tells the reader 'this book is available offline' when arbitrary chapters are missing; they discover it on a plane when a chapter 404s into the SPA error page.

FIX PROMPT

In frontend/src/lib/offlineBooks.svelte.ts download() count successful cache.put calls; if any chapter or the book detail failed, either return false without recording the book (and surface a retry) or record the real cached count and show a partial/retry state. At minimum, don't push the metadata entry when successCount < urls.length.

38 Auto-applied PWA update reloads the page mid-input on most routes

MEDIUM

`frontend/src/lib/pwa.svelte.ts:104`

WHAT

`#applyWhenSafe` treats only `/books/[slug]/[order]` and `/sermons/[slug]` as unsafe; on every other route a waiting worker triggers `SKIP_WAITING` and an immediate `location.reload()`. That includes the settings page `mid-display-name-edit`, the search page with a typed query, an admin form, or the notebook.

WHY IT MATTERS

A deploy landing while a reader is typing silently reloads the tab and destroys their input; 'not mid-chapter' is far narrower than 'safe to reload'.

FIX PROMPT

In frontend/src/lib/pwa.svelte.ts broaden the safety check: skip the reload when any input/textarea/contenteditable is focused or dirty (`document.activeElement`), and add `settings/admin/search` to the unsafe set (or invert to an allowlist of passive routes). Migrate the page import from `$app/stores` to `$app/state` while there.

F. Frontend UX, accessibility & SEO

Error states, focus management, RTL correctness, and the per-locale SEO story on the public reading surfaces.

39 Library search has no error state — API failure shows stale results plus an unhandled rejection

HIGH

frontend/src/routes/search/+page.svelte:569

WHAT

runSearch is try { ... } finally { loading = false } with no catch, invoked fire-and-forget from the debounce timer. When the API call rejects, the rejection is unhandled, loading flips off, and the page keeps rendering the previous query's hits and counts under the new query text with no message.

WHY IT MATTERS

A reader on a flaky connection types a query, the spinner disappears, and they see results for a different search presented as the answer — a silent lie — plus console noise.

FIX PROMPT

In frontend/src/routes/search/+page.svelte add a catch to runSearch that (when the token is current) clears hits/counts and sets a searchError state, and render an error panel with a retry button, following the loadError pattern BooksShelf.svelte already uses. Add the new message key to all catalogues and run npm run sync:catalogues.

40 In-book search drawer is aria-modal but has no focus trap

HIGH

frontend/src/lib/components/SearchDrawer.svelte:132

WHAT

The panel is role='dialog' aria-modal='true' but nothing constrains Tab — unlike its sibling TocDrawer and CommandPalette, which use a focus trap. Focus moves into the input on open but Tab then walks out of the drawer into the page content hidden behind the scrim.

WHY IT MATTERS

Screen-reader and keyboard users are told the rest of the page is inert while it is actually reachable, breaking the modal contract and letting keyboard users escape into invisible content.

FIX PROMPT

In frontend/src/lib/components/SearchDrawer.svelte apply the existing focusTrap action to the .search-panel (as CommandPalette.svelte does), passing { onEscape: close }, and remove the now-redundant manual Escape handling. Verify Tab/Shift-Tab cycle within the panel and focus returns to the opener on close.

41 Unknown topic and plan slugs render a generic error instead of the designed 404

MEDIUM

frontend/src/routes/topics/[slug]/+page.ts:22; frontend/src/routes/plans/[slug]/+page.ts:22

WHAT

Both loads call the API directly without the orNotFound helper that books, chapters, sermons and authors all use to translate an ApiError 404 into SvelteKit's error(404). A mistyped or unpublished slug surfaces as a generic 'something went wrong, try again' page instead of the not-found page with daily picks.

WHY IT MATTERS

Readers hitting a stale topic/plan link get a useless retry for a page that will never exist, and the 404-only picks section never renders; 404 semantics are inconsistent across otherwise-identical detail routes.

FIX PROMPT

In topics/[slug]/+page.ts and plans/[slug]/+page.ts wrap the fetch in the existing helper: import { orNotFound } from '\$lib/loadHelpers' and return await orNotFound(() => getTopic(params.slug, getLang())) (same for getPlan), matching books/[slug]/+page.ts.

42 feed.xml prerender crashes on any work with a missing created_at

MEDIUM

frontend/src/routes/feed.xml/+server.ts:38

WHAT

The sort explicitly anticipates undated rows ('API deploy race: keep them, sort undated last'), but entryXml then does `new Date(it.date).toISOString()`. For an undefined or "" date that is an Invalid Date and `.toISOString()` throws a `RangeError`, so the very rows the guard keeps crash the feed's prerender and fail the whole static build.

WHY IT MATTERS

One backend deploy that briefly serves works without `created_at` turns a cosmetic data gap into a hard web-build failure — the exact race the comment says the file is designed to survive.

FIX PROMPT

In frontend/src/routes/feed.xml/+server.ts make entryXml tolerate a bad date: `const d = new Date(it.date); const updated = isNaN(d.getTime()) ? new Date(0).toISOString() : d.toISOString()`; and apply the same guard to the feed-level updated. Align the comment with whichever behaviour you pick.

43 Physical CSS in scoped styles breaks RTL (Arabic) reading surfaces

MEDIUM

frontend/src/routes/notebook/+page.svelte:310; sermons/[slug]/+page.svelte:503; authors/[slug]/+page.svelte:444

WHAT

`rtl.test.ts` only scans for physical Tailwind classes, so scoped `<style>` blocks slip through: notebook highlight cards mix logical `border-s-2` with a physical `border-left-color` (the colour edge vanishes in RTL); `.text-card`, `.author-quote` and bio blockquotes draw their rule on the wrong side of Arabic prose; sermon outline items use `text-align:left` and the progress bar grows from the wrong end.

WHY IT MATTERS

Arabic is a routed, advertised RTL locale; these details make the reading surfaces look subtly broken there while staying invisible to every LTR locale and to the existing test.

FIX PROMPT

Across frontend/src convert physical properties in reader-facing scoped styles to logical ones (`border-left -> border-inline-start`, `padding-left -> padding-inline-start`, `text-align:left -> start`, `transform-origin` fixes for RTL), starting with notebook, sermons and authors pages, and extend `rtl.test.ts` to also grep `<style>` blocks for `border-left|border-right|text-align:\s*(left|right)|padding-(left|right)`.

44 Hardcoded English meta-description fallbacks on localized book and sermon pages

MEDIUM

frontend/src/routes/books/[slug]/+page.svelte:57; sermons/[slug]/+page.svelte:136

WHAT

The book page falls back to ``${title} by ${author}`` — free to read on Ochorus. — and the sermon page to a similar English literal. These pages prerender per locale (`ar/es/sw/lg/pt/uk`), so a Swahili work without a description ships an English meta and `og:description` at its `/sw` URL. The author page already solved this with a localized `t('author.metaFallback')`.

WHY IT MATTERS

Search snippets and social cards for localized pages appear in the wrong language, which looks broken to readers and undercuts the carefully-built hreflang story.

FIX PROMPT

Add localized fallback messages (`book_metaFallback`, `sermon_metaFallback`) to all eight catalogues, run `npm run sync:catalogues`, and use them with `%title%/%name%` interpolation in `books/[slug]/+page.svelte` and `sermons/[slug]/+page.svelte`, following the `author.metaFallback` pattern.

G. Engineering infrastructure & developer experience

The CI gate, build reproducibility, and test coverage that a fast-moving multi-session repo depends on.

45 The auth boundary (SupabaseJWTAuthentication) has zero tests

HIGH

backend/accounts/authentication.py:27

WHAT

The 120-line SupabaseJWTAuthentication class — JWKS fetch, JWT decode, the documented 'bad token resolves to anonymous, never a 500' contract, and `_get_or_create_user` — has no test anywhere; accounts/tests.py covers only Me/OriginUrl/IsAdminUser.

WHY IT MATTERS

This is the security boundary for every authenticated and admin endpoint; a regression (an expired-token path that raises instead of resolving to anonymous, or a user-creation bug) ships silently since CI proves nothing about it.

FIX PROMPT

Add backend/accounts/tests_authentication.py covering SupabaseJWTAuthentication: a valid RS256 token (mock the JWKS client with a locally generated keypair) authenticates and creates/reuses the user; expired, malformed, wrong-audience and absent tokens resolve to anonymous without raising. Use unittest.mock.patch so no network is touched.

46 No Python linter or formatter exists or runs in CI

HIGH

backend/pyproject.toml:27; .github/workflows/ci.yml

WHAT

The backend has no ruff/flake8/black/mypy configuration anywhere, and ci.yml runs only manage.py test and the migration check — no static-analysis gate at all on Python code.

WHY IT MATTERS

With many parallel AI sessions squash-merging to main fast, unused imports, dead code, shadowed names and style drift accumulate unchecked — exactly the failure mode a fast multi-agent repo needs a linter to catch cheaply.

FIX PROMPT

Add ruff to the backend: put ruff in the dev dependency group in pyproject.toml, add a [tool.ruff] section (target py312, select E,F,I,B,DJ), fix or ignore existing violations file-by-file, then add a 'Backend — lint (ruff)' step to ci.yml right after uv sync --frozen. Mirror with ESLint+Prettier on the frontend.

47 No browser E2E or smoke test; SPA runtime paths are never executed in CI

MEDIUM

frontend/package.json:14

WHAT

There is no Playwright/Cypress anywhere; the only integration coverage is the CI prerender build fetching the seeded API, which exercises public GET endpoints at build time but nothing in a browser — hydration, the reader, client-side routing, auth, offline/PWA, or any authenticated endpoint contract (library-admin.ts is never checked against the real API).

WHY IT MATTERS

Frontend and backend can drift on any authenticated or mutating endpoint (rename a DRF field, change a payload shape) with all of CI green; the bug surfaces only when a user or the admin hits it in production.

FIX PROMPT

Add Playwright with a small smoke suite run in CI after the build: serve build/ honoring the 200.html fallback, then assert the home page hydrates, a book page renders chapter HTML, client navigation to a chapter works, and search returns results against the locally seeded API. Reuse the backend already running on port 8000.

48 The API container runs as root and has no .dockerignore

MEDIUM

`backend/Dockerfile:19`

WHAT

The Dockerfile never switches to a non-root user, so unicorn and the preDeploy manage.py release run as root. There is also no backend/.dockerignore, so COPY . . ingests __pycache__, any local .env, sqlite files and .venv when built locally.

WHY IT MATTERS

Root-in-container needlessly widens the blast radius of any Django/dependency RCE, and a locally built image can leak a developer's .env secrets into image layers.

FIX PROMPT

*In backend/Dockerfile add a non-root user after the final uv sync (useradd app; chown -R app /app; USER app), verifying collectstatic output stays readable, and add backend/.dockerignore with .env, __pycache__/, *.pyc, .env*, *.sqlite3. Confirm manage.py release still works as the non-root user on a Render preview.*

49 Same-commit deploy race lets the web build prerender stale API content

MEDIUM

`render.yaml:79; frontend/svelte.config.js:52`

WHAT

A push touching backend/library/fixtures/** auto-deploys the API and, via buildFilter, rebuilds the web service concurrently. The web build's prerender and fetch-live-locales.mjs hit the production API, which may still be serving the previous version while the API's Docker build runs — so the static site can bake old content (svelte.config.js already excuses routes that 'lag a simultaneous deploy').

WHY IT MATTERS

The rebuild that exists specifically to freshen prerendered content can silently ship the stale content it was meant to replace, with no signal; the next unrelated deploy quietly fixes it.

FIX PROMPT

Make the web build verify its content version: expose an API content fingerprint (e.g. /api/health/ returning the release git SHA) and add a prebuild check comparing it to the building commit, failing with 'API still serving previous release — retry the deploy' on mismatch. Alternatively replace the buildFilter trigger with an explicit web deploy-hook call at the end of manage.py release.

50 The content fixture is ~75 MB and grows with whole-file rewrites per edit

MEDIUM

`backend/library/fixtures/content/`

WHAT

fixtures/content/ is ~75 MB across 234 JSON files (largest 7.2 MB, ten over 1 MB); the git pack is already ~46 MB. Every translation or english-qa fix rewrites an entire multi-MB JSON file, the roadmap (8 locales x 65+ books plus contemporized editions) multiplies this, and each fixture commit triggers a full frontend prerender rebuild.

WHY IT MATTERS

Clone/checkout/CI time and repo size grow superlinearly with content velocity (dozens of commits per month), and GitHub becomes slow and hostile to review well before hard limits hit.

FIX PROMPT

Produce a design note projecting fixture growth and CI-time, then evaluate (a) a one-file-per-chapter diff-friendly layout so edits touch kilobytes not megabytes, (b) Git LFS, or (c) object-store-backed content with only metadata in git. Constraints: manage.py release must still seed from files in the Docker build context, and content_fixtures.py + tests_fixture + regen_fixture.py define the current contract.

Recommended order of work

Fifty items is a backlog, not a sprint. The order below is by **reader-facing risk per unit of effort**: correctness and data-safety bugs that silently harm real readers or violate the app's own promises come first; the deep-but-large refactors come last. Numbers in brackets reference the findings above.

Wave 1 — Stop the silent harm (this week)

Small, high-certainty fixes that either leak data, mislead readers, or break an invariant the app advertises. Most are a few lines plus a test.

- **Admin unverified-email takeover [01]** — the one true security hole; require an email_verified claim before any admin decision. Ship first.
- **Service worker caches authenticated /api/ [34] and session-expiry skips the wipe [35]** — together these leak one reader's private data to the next person on a shared device; fix both before anything else user-facing.
- **Bookmarks wiped but never synced [36]** — guaranteed silent data loss on every sign-out; the one-line stopgap (drop BOOKMARKS_KEY from the wipe list) buys time for the real sync.
- **New chapters keep an approved book 'reviewed' [24]** — unreviewed AI scripture shown as native-approved; flip source_type back on any chapter write.
- **Merge 500s on malformed JSON [08]** — one bad field aborts a reader's whole first-sign-in reconciliation; add the missing type guards.
- **Search shows stale results on API error [42]** — a silent wrong answer on the core feature; add the missing catch and error panel.

Wave 2 — Data integrity & abuse resistance (next 2–3 weeks)

Bugs that corrupt stored data under concurrency or let a single request amplify into unbounded work. Each needs a little more care and a concurrency/limit test.

- **Throttle + cap the reading merge [09]** — no /api/reading/ endpoint is rate-limited and most merge lists are uncapped; add a per-user throttle and per-type caps.
- **Make merges atomic [10] and lock plan-progress [11]** — wrap MergeView in a transaction and use select_for_update so a completed day is never lost or half-applied.
- **fetch_chapter negative-cache [28]** — a single timeout can silently ship a whole book of scripture-less AI translation; cache only 404s and retry transient failures.
- **Re-import republishes pulled books [26] & approvals lost on rebuild [27]** — make is_published/source_type create-only, and persist approvals into the fixture so a rebuild can't un-approve.
- **Reject junk identifiers [12][13][14]** — negative chapter orders, 300-char slugs, and unbounded notes all become 500s or storage abuse; validate at the boundary.
- **seed_plans atomicity [25]** — a mid-seed crash leaves a permanently empty plan the idempotent command can never repair.

Wave 3 — Guardrails so regressions can't recur (parallel track)

Infrastructure that would have caught several of the bugs above and will catch the next batch. Best done alongside the fixes, not after.

- **Tests for the auth boundary [48]** — the security-critical class has zero coverage; add it before touching finding 01.
- **Ruff + ESLint/Prettier in CI [49]** — a fast-moving multi-session repo has no static-analysis gate at all; cheap and high-leverage.
- **A Playwright smoke suite [50]** — nothing exercises the SPA in a browser or checks the real API contract; a handful of tests closes the frontend/backend drift gap.
- **Non-root container + .dockerignore [45]** — stop leaking .env into image layers and shrink the RCE blast radius.
- **Fix the deploy-race stale prerender [46]** — add an API content fingerprint the web build verifies, so content deploys can't silently ship old text.

Wave 4 — Performance & polish (as capacity allows)

Efficiency and consistency wins on the prerender hot paths, plus the accessibility, RTL, SEO and pipeline items. None are emergencies; batch them by file to keep diffs small.

- **Query hot paths [17][18][19][23]** — memoize the author serializer, defer chapter bodies for the TOC, throttle the search log; measurable build-time and latency wins.
- **Payload consistency [15][16]** — restore word_count and topic chips everywhere the shelf serializer is reused.
- **Accessibility & RTL [43][44][45][47]** — focus-trap the search drawer, 404 the unknown slugs, guard feed.xml, and convert physical CSS to logical for Arabic.
- **Per-locale SEO [42-adjacent, meta fallbacks] [46-F]** — localized meta descriptions so non-English pages stop shipping English snippets.
- **The big refactor: fixture growth [46-G]** — the ~75 MB whole-file-rewrite fixture is the one architectural item; scope it as a design note before the content roadmap makes it urgent.

One closing note: this is an unusually well-tended codebase — complete i18n catalogues across eight locales, a thoughtful hreflang/canonical story, consistent JSON-LD escaping, correct object-scoping on every reading endpoint (no IDOR found), and production security headers already in place. The findings above are the sharp edges on a solid foundation, not signs of neglect.